

● КосмоХакатон 2026 · федеральный проект «Кадры для космоса»

Описание данных



Автономное управление спутниковой группировкой



МИНОБРНАУКИ
РОССИИ



МИРЭА —
Российский
технологический
университет



РОСКОСМОС

6 хакатонов. Один космос.

КОСМОХАКАТОН.РФ

Состав комплекта и начало работы

В исходном сценарии находятся начальные состояния аппаратов, плановые задания, внешние условия и известные интервалы недоступности. Новые сообщения передаются отдельно. Координаты спутников, потоки отдельных пакетов и работа приёмных станций внутри сеанса не рассчитываются: доступность связи уже задана рядами.

ФАЙЛ	СОДЕРЖАНИЕ
data/P01_intro.json	16 аппаратов, 48 шагов, четыре часа, 18 заданий.
data/P02_shift.json	48 аппаратов, 288 шагов, сутки, 1208 заданий.
data/P03_energy.json	Те же задания, что в P02; солнечная мощность составляет 40% от P02, начальный заряд первых 12 аппаратов — 38%.
data/P04_demand.json	Аппараты и внешние условия P02, очередь из 8120 заданий. Исходные задания P02 входят в неё без изменения.
examples/events_demo.json	Пример последовательности новых сообщений для P02.
model/resource_env.py	Расчёт энергии, температуры, допуска действий и выполнения заданий.
model/operations.py	Приём сообщений, выполнение шагов, копирование состояния и воспроизведение журнала.

Для начала достаточно открыть P01, создать расчёт через Session, получить состояние, передать действия на один шаг и посмотреть изменения. Затем подключается собственный планировщик. Библиотеки работают на Python 3.10 и новее без сторонних зависимостей. Использование Python-файлов необязательно: реализация на другом стеке должна соблюдать описанные правила.

Формат исходного сценария

Кодировка JSON — UTF-8. Версия schema_version — cosmo-B-ops-1.0.

РАЗДЕЛ	СОДЕРЖАНИЕ
meta	id — идентификатор сценария; title — название. Метаданные не влияют на решения и физический расчёт.
time	step_s — длительность шага; steps — число шагов.
satellites	Список аппаратов с характеристиками и начальными состояниями.
model	Параметры переходов состояния и допуска действий.
environment	Внешние условия по каждому аппарату на всех шагах.



РАЗДЕЛ	СОДЕРЖАНИЕ
jobs	Плановые задания и допустимые исполнители.
failures	Известные с начала смены интервалы недоступности; список может быть пустым.

Все плановые задания, внешние ряды и записи failures исходного файла известны планировщику с начала смены. Новые задания и ограничения становятся известны при получении соответствующего сообщения. В объекте расчёта нет очереди будущих нераскрытых сообщений.

Время

step_s = 300 секунд. Шаг k соответствует интервалу $[k \times 300; (k + 1) \times 300)$. Индексация начинается с нуля. Операция на шаге k заканчивается на границе k + 1.

Перед шагом 72 прошло шесть часов и выполнены шаги 0–71. В этот момент можно принять сообщение и выбрать действия на 06:00–06:05. Приостановка перед шагом не меняет состояние. Время относительное; для отображения используется время от начала смены.

Допустимы сценарии с 1–48 аппаратами и 1–288 шагами. Основные файлы используют 16 или 48 аппаратов и 48 или 288 шагов. Модельное время может рассчитываться ускоренно; соединение с часами реального времени не требуется.

Характеристики аппарата

ПОЛЕ	ЗНАЧЕНИЕ
id	Уникальная непустая строка.
capacity_wh	Ёмкость батареи в Вт·ч; в наборе 100, 120 или 140.
initial_soc_pct	Начальный заряд, 0–100%.
initial_temp_c	Начальная температура контролируемого оборудования, °C.
base_w	Постоянная мощность служебных систем, 18 Вт.
heater_w	Мощность включённого нагревателя, 30 Вт.
calibration_w	Дополнительная мощность калибровки, 20 Вт.
downlink_w	Дополнительная мощность передачи на Землю, 90 Вт.
relay_w	Дополнительная мощность ретрансляции, 65 Вт.
initial_calibration_age_steps	Число шагов после последней калибровки на начало расчёта. Целое неотрицательное число.

Начальная энергия: $\text{capacity_wh} \times \text{initial_soc_pct} / 100$. Мощность операции добавляется к постоянному потреблению и, при необходимости, к мощности нагревателя. После старта энергия, температура и возраст калибровки определяются действиями; редактирование начальных значений не меняет уже идущую смену.

Параметры модели

ПОЛЕ	ЗНАЧЕНИЕ В НАБОРЕ
reserve_soc_pct	30% — резерв при допуске калибровки и заданий.
critical_soc_pct	20% — порог критического дефицита для отчётности.
charge_efficiency	0,92 — коэффициент зарядки.
discharge_efficiency	0,95 — коэффициент разрядки.
thermal_tau_s	1800 с — постоянная времени теплового расчёта.
thermal_gain_c_per_w	0,20 °C/Вт — влияние нагрузки на равновесную температуру.
heater_below_c	5 °C — нагреватель включается при температуре строго ниже порога.
payload_min_c, payload_max_c	От 5 до 45 °C включительно — допустимые температуры калибровки и заданий.
charge_min_c, charge_max_c	От 0 до 45 °C включительно — температуры, при которых накапливается избыток энергии.
calibration_valid_steps	48 шагов — срок действия калибровки.
downlink_parallel_limit	2 — максимальное число одновременных передач на Землю.

Значения считываются из сценария. В сравнении стратегий они одинаковые. Параметры задают расчётные условия кейса и не являются характеристиками конкретного действующего аппарата.

Внешние условия

environment — объект, ключами которого являются идентификаторы аппаратов. Для каждого аппарата есть четыре массива длиной time.steps; элемент с индексом k используется на шаге k.

ПОЛЕ	СОДЕРЖАНИЕ
solar_w	Доступная солнечная мощность в ваттах, неотрицательная.
thermal_target_c	Температурный параметр внешнего воздействия, °C.
downlink_available	true или false: доступна ли передача на Землю на этом шаге.



ПОЛЕ	СОДЕРЖАНИЕ
relay_available	true или false: доступна ли ретрансляция.

Солнечная мощность используется в балансе энергии; тепловое воздействие задаётся отдельным рядом. Прямого правила «больше солнечной мощности — выше температура» в модели нет. При умножении солнечной мощности в отдельном эксперименте изменяется только solar_w.

Для ретрансляции не требуется строить пары спутников или маршруты: задание занимает ресурс одного допустимого исполнителя на шаг. Его наличие в списке исполнителей и доступность relay_available задают применимость операции.

Задания

ПОЛЕ	СОДЕРЖАНИЕ
id	Уникальная непустая строка.
kind	downlink или relay.
release_step	Первый разрешённый шаг выполнения.
deadline_step	Граница, к которой задание должно быть завершено.
work_steps	Число успешных рабочих шагов, необходимое для завершения.
eligible_satellites	Непустой список допустимых исполнителей. Для downlink — ровно один аппарат.
priority	1, 2 или 3; 3 — высший приоритет.
value_usd	Выручка за полное завершение в срок, в долларах по условиям сценария.

Пример: release_step = 2, deadline_step = 5, work_steps = 2. Нужно выполнить два рабочих шага среди 2, 3 и 4. Можно выполнить шаги 2 и 4: прогресс сохраняется, завершение на границе 5 своевременно. Начинать операцию на шаге 5 уже поздно.

Один аппарат выполняет одну операцию на шаг. Над одним заданием на шаге работает один аппарат. Для relay допустима смена исполнителя между шагами. Для downlink исполнитель не меняется. Калибровка принадлежит аппарату, а не заданию.

Приоритет не является множителем стоимости. Высокая стоимость также не означает приоритет 3. Задание оплачивается один раз после полного своевременного выполнения. Частичное выполнение или срыв срока дают нулевую выручку. Дополнительного почасового дохода нет.

Известные периоды недоступности

Каждая запись failures содержит satellite_id, start_step, end_step. Интервал [start_step; end_step) включает начало и исключает конец. Пересекающиеся интервалы действуют совместно.

Во время недоступности аппарат выполняет ожидание: служебное потребление, нагреватель и изменение температуры продолжают. Задания и калибровка недоступны. Прогресс заданий и возраст калибровки не сбрасываются; возраст увеличивается с каждым шагом ожидания. После окончания интервала выполняются обычные проверки ресурсов и контактов.

Сообщения во время смены

Сообщение передаётся до выбора действий текущего шага. Общие поля: id, at_step, type. id должен быть новым непустым идентификатором; at_step должен совпадать с текущим незавершённым шагом. Сообщения с прошлым или будущим at_step не применяются. Несколько сообщений на одном шаге обрабатываются в порядке получения до выполнения действий.

Поступление заданий: add_jobs

Дополнительное поле jobs содержит непустой список заданий описанного выше формата. Идентификаторы не должны повторять уже существующие. release_step нового задания не меньше at_step; конечный срок находится внутри смены. Поступление сообщения и начало допустимого выполнения могут различаться.

```
{
  "id": "E-NEW-JOB",
  "at_step": 72,
  "type": "add_jobs",
  "jobs": [{
    "id": "NEW-RELAY-01",
    "kind": "relay",
    "release_step": 72,
    "deadline_step": 80,
    "work_steps": 3,
    "eligible_satellites": ["S08", "S10", "S12"],
    "priority": 3,
    "value_usd": 40
  }]
}
```

Недоступность аппаратов: satellite_outage

Дополнительные поля — satellite_ids и end_step. На интервале [at_step; end_step) выбранные аппараты недоступны для калибровки и заданий. Окончание интервала известно из сообщения.

```
{
  "id": "E-OUTAGE",
  "at_step": 74,
  "type": "satellite_outage",
  "satellite_ids": ["S08", "S10"],
  "end_step": 90
}
```

Отмена сеансов: close_downlink

Поля — `satellite_ids` и `end_step`. На интервале `[at_step; end_step)` доступность downlink выбранных аппаратов становится `false`. Ретрансляция, солнечная мощность и тепловые условия этим событием не меняются. После конца интервала используется исходное расписание контактов с учётом остальных полученных ограничений; доступность не устанавливается безусловно в `true`.

```
{
  "id": "E-CONTACT",
  "at_step": 144,
  "type": "close_downlink",
  "satellite_ids": ["S01", "S02"],
  "end_step": 156
}
```

Для всех событий начало строго меньше конца, конец не превышает `time.steps`. Списки аппаратов непустые, без повторов. Поля отдельного сообщения строго соответствуют одному из трёх типов. При ошибке отклоняется всё сообщение, включая пакет новых заданий; частичного применения нет.

Пример последовательности `events_demo.json` содержит `schema_version = cosmo-B-events-1.0`, `base_scenario = P02_shift` и массив `events`. Это открытый пример для разработки. Интерактивный ввод принимает один объект события, без этой оболочки. На проверке жюри передаёт отдельные сообщения, не весь будущий сценарий событий.

Изменение состояния на шаге

Обозначения: E — энергия в начале шага, Вт·ч; C — ёмкость, Вт·ч; T — начальная температура, °C; $dt = 3600$ с; S — солнечная мощность; P — дополнительная мощность операции. Для ожидания $P = 0$.

Нагреватель потребляет $H = \text{heater_w}$, если $T < \text{heater_below_c}$, иначе $H = 0$. Полная нагрузка $L = \text{base_w} + H + P$. Баланс мощности $b = S - L$.

Изменение энергии:

- при $b \geq 0$ и начальной температуре в диапазоне зарядки: $\Delta E = b \times dt / 3600 \times \text{charge_efficiency}$;
- при $b \geq 0$ вне диапазона зарядки: $\Delta E = 0$;
- при $b < 0$: $\Delta E = b \times dt / 3600 / \text{discharge_efficiency}$.

$E_{\text{raw}} = E + \Delta E$. Следующая энергия $E_{\text{next}} = \min(C, \max(0, E_{\text{raw}}))$. При $E_{\text{raw}} < 0$ фиксируется эпизод нехватки питания. Отдельная модель перезапуска оборудования после него не вводится: продолжается описанный учёт состояния.

Температура:

$$T_{\text{eq}} = \text{thermal_target_c} + \text{thermal_gain_c_per_w} \times L$$

$$T_{next} = T_{eq} + (T - T_{eq}) \times \exp(-dt / thermal_tau_s)$$

Нагреватель выбирается по температуре в начале шага. Условия зарядки также проверяются по начальной температуре. Для допуска полезной операции проверяются и начальная, и расчётная конечная температура.

Допуск и порядок выполнения действий

В начале шага применяются полученные сообщения. Планировщик получает актуальное состояние и выбирает действия.

Перед калибровкой или выполнением задания проверяются доступность аппарата, энергия в начале и расчётном конце операции не ниже резерва, а также обе температуры в рабочем диапазоне. При задании дополнительно проверяются наличие задания, отсутствие завершения, срок, допустимый исполнитель, контакт и действующая калибровка.

При возрасте калибровки, равном `calibration_valid_steps` или выше, выполнение задания запрещено до калибровки. Калибровка занимает один полный шаг. После успешной калибровки возраст становится нулём; на любом другом шаге увеличивается на один.

Запрошенные действия рассматриваются в порядке сортировки идентификаторов аппаратов по возрастанию. Из допустимых действий резервируется общий ресурс: не более двух `downlink` и не более одного исполнителя одного задания на шаге.

Последующие конфликтующие действия отклоняются. Планировщик должен сам согласовывать назначения, а не рассчитывать на отбор конфликтов библиотекой.

Отклонённое действие заменяется ожиданием. Энергия и температура рассчитываются для фактически выполненного действия. Причина отклонения сохраняется. Успешный шаг задания уменьшает оставшуюся работу на один; завершение отмечается на границе следующего шага. Если запись действия для аппарата отсутствует, выполняется ожидание.

Резерв 30% ограничивает допуск калибровки и заданий. Даже во время ожидания аппарат расходует энергию, поэтому заряд может опуститься ниже резерва. Причину нужно разбирать по условиям и истории решений. В расчётном модуле сравнение энергии с резервом использует численный допуск $1e-9$ Вт·ч; температура сравнивается с включёнными границами диапазона.

Расчётная библиотека

Основной интерфейс находится в `model/operations.py`.

ВЫЗОВ	НАЗНАЧЕНИЕ
<code>Session(scenario, run_metadata=None)</code>	Создать отдельную смену из исходного сценария; при необходимости передать сведения об алгоритме и цели.



ВЫЗОВ	НАЗНАЧЕНИЕ
<code>session.observation()</code>	Получить текущий шаг, копию состояний и заданий с прогрессом, доступность аппаратов.
<code>session.env.s</code>	Исходный план с уже применёнными сообщениями. Будущих нераскрытых событий здесь нет.
<code>session.env.can_execute(sid, action)</code>	Проверить действие для одного аппарата; общий лимит и конфликты окончательно проверяются при выполнении шага. Возвращает допустимость, причину и дополнительную мощность.
<code>session.apply_event(event)</code>	Принять одно сообщение на текущем шаге без сброса состояния.
<code>session.advance(actions)</code>	Применить словарь действий, записать журнал, перейти на следующий шаг.
<code>session.fork()</code>	Получить независимую копию текущего состояния и истории для сравнения.
<code>session.summary()</code>	Получить показатели исполненной части смены.
<code>session.result()</code>	Получить исходный сценарий, сообщения, команды, показатели и подробный журнал.
<code>replay_episode(scenario, events, commands, until_step=None)</code>	Повторить выполненные действия с теми же сообщениями до указанной границы; без <code>until_step</code> — до конца смены.

Пример подключения из приложения в корне комплекта:

```
from model.resource_env import load
from model.operations import Session

scenario = load("data/P01_intro.json")
session = Session(scenario, run_metadata={
    "goal": "priority",
    "algorithm": "team-planner",
    "version": "1.0",
    "parameters": {}
})

state = session.observation()
# Здесь приложение вызывает собственный планировщик.
# Пустой словарь означает ожидание всех аппаратов на одном шаге.
session.advance({})
print(session.observation()["step"]) # 1
```

Действия передаются так:

```
{
  "S01": {"action": "job", "job_id": "JOB-0001"},
  "S02": {"action": "calibrate"},
  "S03": {"action": "idle"}
}
```

Неизвестный аппарат, неизвестный тип действия или неправильная структура словаря отклоняются до исполнения шага. Структурно корректная команда задания, не прошедшая предметные ограничения, записывается как отклонённая с выполнением ожидания.

При сравнении стратегия продолжения и её внутреннее состояние принадлежат соответствующей ветви. События передаются в обе ветви в одном порядке. Внутреннее состояние алгоритма команда копирует или восстанавливает самостоятельно; `fork()` копирует состояние среды и уже исполненную историю.

Показатели

Показатели на промежуточном шаге относятся к уже выполненной части смены. Ожидаемый результат будущего плана показывается отдельно от факта.

ПОЛЕ SUMMARY	ЗНАЧЕНИЕ
steps_executed	Число выполненных шагов.
jobs_total	Число всех известных заданий, включая поступившие.
jobs_completed	Число завершённых заданий.
jobs_due	Число заданий, чья граница срока уже наступила.
jobs_due_missed	Число таких заданий, оставшихся незавершёнными.
critical_jobs_due	Число заданий приоритета 3 с наступившим сроком.
critical_jobs_completed_on_time	Сколько из них завершено в срок.
revenue_usd	Сумма value_usd всех завершённых в срок заданий.
blocked_command_count	Число запрошенных, но отклонённых команд.
below_reserve_satellite_steps	Число пар «аппарат — шаг» с конечным зарядом ниже резерва.
critical_soc_satellite_steps	Число пар с конечным зарядом ниже критического порога.
brownout_satellite_steps	Число пар с расчётной энергией до ограничения $E_{\text{raw}} < 0$.
minimum_soc_pct	Минимальный заряд среди начальных и рассчитанных состояний.
terminal_soc_pct	Заряд каждого аппарата на текущей конечной границе расчёта.
work_steps_in_missed_jobs	Выполненные рабочие шаги в заданиях, срок которых наступил, но полного завершения нет.

Доля выполнения приоритетных заданий равна отношению двух соответствующих счётчиков. При нулевом знаменателе показатель отсутствует, а не принимается за 0% или 100%. Загрузка аппарата — доля выполненных шагов заданий за исполненный период; ожидание и калибровка учитываются отдельно. При нуле выполненных шагов доли отсутствуют.

В конце смены сроки всех полученных заданий находятся внутри расчётного периода, поэтому учёт охватывает весь набор. Удаление невыполненных заданий из знаменателя или переоценка их стоимости после исполнения не допускаются.

Выгрузка результата

JSON результата содержит:

- `schema_version` = `cosmo-B-ops-result-1.0`;
- `initial_scenario` — полный сценарий на начало этой смены;
- `events` — только фактически полученные сообщения в порядке получения;
- `commands` — запрошенные действия с полями `step`, `satellite_id`, `action` и `job_id` для задания;
- `steps_executed` — число исполненных шагов;
- `summary` — перечисленные сводные показатели;
- `run_metadata` — цель, название и версия алгоритма, параметры; при наличии переключений — их шаги и значения, при ветвлении — идентификатор исходного расчёта и шаг развилки.

Для каждой пары «шаг — аппарат» в `commands` допускается одна запись, пропуск означает ожидание. Команды на ещё не выполненные шаги в экспорт фактической истории не включаются. Будущий предлагаемый план можно выгружать отдельным полем.

Библиотека также добавляет `initial_scenario_hash` и `trace`. В `trace` сохраняются исходные и конечные энергия и температура, запрошенное и выполненное действие, причина отклонения, возраст калибровки, мощность и завершение задания. Для собственной реализации подробный `trace` в файле необязателен, если эти сведения доступны в сервисе и восстанавливаются по обязательным полям. Сводка обязана соответствовать журналу команд.

`run_metadata` заполняет приложение команды: библиотека не определяет цель и не выбирает алгоритм. Дополнительные поля результата разрешены, если не подменяют обязательные. Состояния внутри алгоритма рассчитываются без промежуточного округления; библиотека округляет числовые значения журнала и сводки до шести знаков.

Воспроизведение выгрузки из корня комплекта:

```
python model/operations.py --result result.json --output replay.json
```

Воспроизведение заранее сохранённых команд с открытым примером событий:

```
python model/operations.py --scenario data/P02_shift.json --events examples/events_demo.json  
--commands commands.json --output replay.json
```

`commands.json` — массив команд или объект с полем `commands`. Эти команды проверяют расчёт последствий уже выбранных действий; планировщик при воспроизведении не запускается. При сохранении на границе шага экспорт может включать сообщение, уже полученное на этой границе, даже если действие после него ещё не выполнено.

Проверка входа

Идентификаторы непустые и уникальные в своих списках, ссылки разрешаются. Числа конечные; NaN и бесконечности не допускаются. Времена и длительности — целые числа, логическое значение не заменяет числовой индекс. Ёмкость положительна, мощность и стоимость неотрицательны, начальный заряд находится в диапазоне 0–100%. Массивы внешних условий имеют нужную длину, доступность задаётся `true/false`.

Для задания выполняется $0 \leq \text{release_step} < \text{deadline_step} \leq \text{steps}$ и $1 \leq \text{work_steps} \leq \text{deadline_step} - \text{release_step}$. Корректные границы ещё не гарантируют выполнимость: её могут ограничивать ресурсы, контакты или другие задания. Новые сообщения не изменяют прошлые действия и не редактируют ранее полученные задания.